

MİMARİ ANALİZ RAPORU

Veri Toplama Mimarisi

CedasISG — "Thundershock" Elektrik Sahası Eğitim Simülasyonu
Unity 6 + PlayFab Client SDK

PlayFab Client SDK

WritePlayerEvent

Tekil Singleton Mimari

Batch / Retry Yok

KVKK/GDPR Katmanı Yok

Depo: D:\CedasISG (git — branch: main)

Rapor Tarihi: 31 Temmuz 2026

Kapsam: Assets/_Project/Scripts altındaki tüm oyun-içi telemetri/analitik akışı, PlayFab entegrasyonu, olay envanteri, depolama ve gizlilik durumu

Yöntem: Tüm proje kaynak kodu (C#), Packages/manifest.json ve ilgili tasarım dokümanları statik olarak taranarak çıkarılmıştır

1. Genel Bakış

CedasISG projesi (ürün adı *Thundershock*), sahadaki elektrik çalışanlarına yönelik bir Unity 6 eğitim simülasyonudur. Projenin veri toplama mimarisi, hazır bir analitik SDK'sı (Firebase, Adjust, AppsFlyer, GameAnalytics, Segment, Unity Gaming Services vb.) kullanmak yerine, doğrudan **PlayFab Client SDK** üzerine inşa edilmiş, **elle yazılmış tek bir telemetri katmanından** oluşur. Tüm veri toplama tek bir merkezi singleton sınıf olan `PlayFabDataManager` üzerinden yürütülür; bu sınıf her oyun-içi olayı sabit bir zarf (*envelope*) içine sarar ve `PlayFabClientAPI.WritePlayerEvent` çağrısıyla doğrudan PlayFab sunucularına gönderir.

Olaylar; seviye/sekans/aksiyon akışını yöneten `SequenceManager`, quiz ekranı `UIQuizPanel`, sürükle-bırak bölgesi `UIDropZone`, saha anketi `SurveyResultTracker` ve giriş ekranı `UILoginPanel` gibi bir avuç oyun-akışı yöneticisinden tetiklenir. Uygulamada reklam, uygulama-içi satın alma, sunucu tarafı (CloudScript) doğrulama/toplama veya kalıcı çevrimdışı önbellek **bulunmamaktadır** — küçük bir bellek-içi kuyruk dışında hiçbir dayanıklılık/yeniden-deneme mekanizması yoktur. Ayrıca projede KVKK/GDPR onayı, opt-out veya veri silme mekanizması yer almamaktadır.

2. Entegre Servisler ve SDK'lar

Servis	Kanıt / Konum	Durum
PlayFab Client SDK	Assets/PlayFabSDK/** (Client, Server, Admin, CloudScript, Events, Economy, Entity vb.)	Aktif kullanımda — yalnızca <code>LoginWithCustomID</code> , <code>GetTitleData</code> , <code>WritePlayerEvent</code> çağrılıyor
PlayFab Editor Extensions	Assets/PlayFabEditorExtensions/**	Yalnızca editör aracı, runtime'da kullanılmıyor
Unity Analytics modülü	Packages/manifest.json → com.unity.modules.unityanalytics	Ölü bağımlılık — kod tarafında hiçbir çağrı yok
Firebase / Adjust / AppsFlyer / Facebook SDK / GameAnalytics / Segment	Assets/Plugins, Packages/manifest.json – tüm depo taranarak arandı	Yok — entegre değil
PlayFab CloudScript (sunucu tarafı fonksiyonlar)	Assets/PlayFabSDK/CloudScript/*	Kullanılmıyor — Assets/_Project içinde hiçbir <code>ExecuteCloudScript</code> çağrısı yok, SDK dosyaları atıl
QR (ZXing)	Assets/Plugins/ZXing, Assets/_Project/Scripts/QR/*	QR tarama özelliği — telemetriyle ilgisi yok

3. Temel Yönetici Sınıflar

Sınıf	Dosya	Sorumluluk
PlayFabDataManager	Assets/_Project/Scripts/PlayFabDataManager.cs	Singleton (<code>DontDestroyOnLoad</code>), <code>MENU.unity</code> sahnesine bağlı. PlayFab girişini (<code>LoginWithPlayer</code> , <code>LoginWithCustomID</code>), whitelist çekimini (<code>FetchWhitelist</code> / <code>FetchTitleData</code> → <code>GetTitleData</code>), bekleyen-olay kuyruğunu, zarf oluşturmayı ve tüm <code>Log*</code> genel metodlarını (<code>LogLevelStarted/Completed</code> , <code>LogSequenceStarted/Completed</code> , <code>LogActionStarted/Completed</code> , <code>LogQuizAnswered</code> , <code>LogQuizSummary</code> — özel/otomatik, <code>LogDragDropAttempt</code> , <code>LogMistakeRecorded</code> , <code>LogSurveyCompleted</code> , <code>LogSessionEnded</code>) barındırır. Gönderim çekirdeği: <code>SendEvent()</code> → <code>SendEventRequest()</code> → <code>PlayFabClientAPI.WritePlayerEvent</code> .
SequenceManager	Assets/_Project/Scripts/SequenceManager.cs	Ana oyun-akışı durum makinesi (seviye → sekans → aksiyon ilerlemesi, puanlama, hatalar, kamera/arayüz orkestrasyonu). Telemetri için en büyük tetikleyici kaynak — <code>StartLevel</code> , <code>EnterSequence</code> , <code>StartNextActionInSequence</code> , <code>OnActionCompleted</code> , <code>OnQuizFailed</code> , <code>CompleteSequenceRoutine</code> , <code>CompleteLevel</code> noktalarında <code>PlayFabDataManager</code> 'ı çağırır.
UIQuizPanel	Assets/_Project/Scripts/UIQuizPanel.cs	Quiz arayüzü; cevap tıklamasında <code>LogQuizAnswered</code> çağırır, yanlışsa <code>SequenceManager.OnQuizFailed</code> 'ı tetikler.
UIDropZone	Assets/_Project/Scripts/UIDropZone.cs	Sürükle-bırak / ekipman yuvası mantığı; yerleştirme denemelerini biriktirir, başarılıda <code>LogDragDropAttempt</code> , yanlış bırakmada <code>LogMistakeRecorded</code> çağırır.
SurveyResultTracker	Assets/_Project/Scripts/SurveyResultTracker.cs	Aktif anket aksiyonu için soru cevaplarını ve fotoğraf-yakalama sonuçlarını biriktiren singleton; <code>EndSession()</code> içinde <code>PlayFabDataManager.LogSurveyCompleted</code> 'ı çağırır. <code>UISurveyPanel.cs</code> tarafından yönlendirilir (<code>BeginSession</code> , <code>RecordAnswer</code> , <code>RecordPhoto</code> , <code>EndSession</code>).
UILoginPanel	Assets/_Project/Scripts/UILoginPanel.cs	Çalışan giriş ekranı; whitelist'i çeker ve <code>PlayFabDataManager.FetchWhitelist</code> / <code>LoginWithPlayer</code> ile kimlik doğrular — sonraki her olayda kullanılan <code>CurrentPlayerId</code> / <code>CurrentPlayerRole</code> burada kurulur.
ActionEventHandler / ActionEventHelper	Assets/_Project/Scripts/ActionEventHandler.cs, ActionEventHelper.cs	Analitikle ilgisi yok — <code>ActionData.onStartEventIDs</code> / <code>onCompleteEventIDs</code> / <code>onFailEventIDs</code> için genel bir sahne-içi VFX/animasyon tetikleme köprüsü.
PlayFabWhitelistData	Assets/_Project/Scripts/PlayFabWhitelistData.cs	ScriptableObject; çalışan whitelist şemasını (<code>playerId</code> , <code>displayName</code> , <code>role</code>) tanımlar — PlayFab Title Data ile eşleşen editör-tarafı kaynak.

4. Veri Akışı / Pipeline

```
Uygulama açılışı (MENU.unity)
→ PlayFabDataManager.Awake()      # singleton kurulur, DontDestroyOnLoad
→ PlayFabDataManager.Start()      # PlayFabSettings.TitleId atanır
→ UILoginPanel.Start() → FetchWhitelist()
    → PlayFabClientAPI.LoginWithCustomID(CustomId="whitelist_reader") # paylaşılan okuyucu hesap
    → PlayFabClientAPI.GetTitleData(key="PlayerWhitelist") → PlayerEntry[]

→ Çalışan kendi ID'sini girer → UILoginPanel.OnLoginClicked()
    → PlayFabDataManager.LoginWithPlayer(entry)
    → PlayFabClientAPI.LoginWithCustomID(CustomId=entry.playerId, CreateAccount=true)
    → OnLoginSuccess: _isLoggedIn=true, _playFabId ayarlanır, FlushPendingEvents()

→ Oyun-içi tetikleyiciler Log*() çağrılarını ateşler (bkz. Bölüm 5)
    → SendEvent(eventName, payload)
        - payload'a sessionId, playerId, role, timestamp eklenir
        - { eventType, clientTimestamp, employeeId, payload } zarfına sarılır
        - giriş yapılmadıysa → _pendingEvents kuyruğuna eklenir (bellek-içi)
        - giriş yapıldıysa → SendEventRequest() → PlayFabClientAPI.WritePlayerEvent (anlık, tekil, batch yok)

→ PlayFab backend (Title-scoped) WritePlayerEvent çağrılarını alır/saklar

→ (Bu depo dışında) ayrı bir KPI/raporlama portalı
    (Caner_UI/thundershock-kpi-wireframe) bu olayları çalışan/yönetici
    panolarında kullanmak üzere tüketir — şu an yalnızca sahte veriyle
    çalışan bir tasarım-taslağı, bu depoda canlı bir tüketici değil
```

Assets/_Project içinde CloudScript veya başka bir sunucu-tarafı fonksiyon çağrısı yoktur — mevcut tüm CloudScript SDK dosyaları kullanılmayan kütüphane koduttur. PlayFab, verinin gittiği **tek** hedeftir; ayrı/özel bir backend bulunmamaktadır.

5. Toplanan Olay (Event) Envanteri

Her olay için zarf yapısı (`PlayFabDataManager.SendEvent` , ~L544–578):

```
{
  "eventType": "<EventName>",
  "clientTimestamp": "<UTC ISO-8601, istemci saati>",
  "employeeId": "<CurrentPlayerId>",
  "payload": { ...olaya özgü alanlar..., "sessionId", "playerId", "role", "timestamp" }
}
```

(`PlayerId` / `timestamp`) isimleri PlayFab tarafından ayrılmış olduğundan, kod içi yoruma göre bilinçli olarak payload seviyesinde farklı biçimde enjekte edilir.)

#	eventType	Tetikleyici	Gönderen (dosya:metot)	Ana payload alanları
1	LevelStarted	Seviye sahnesi yüklenir / <code>StartLevel()</code>	SequenceManager.cs StartLevel() (L159)	levelId, displayName
2	LevelCompleted	Bir seviyedeki tüm sekanslar tamamlanır	SequenceManager.cs CompleteLevel() (L845)	levelId, completed, score, timeSpent, mistakes, completionRate*
3	SequenceStarted	Oyuncu bir sekans butonuna tıklar	SequenceManager.cs EnterSequence() (L252)	sequenceId, levelId, startTime
4	SequenceCompleted	Bir sekanstaki tüm aksiyonlar tamamlanır	SequenceManager.cs CompleteSequenceRoutine() (L748)	sequenceId, levelId, timeSpent, mistakes, completed, startTime, endTime
5	ActionCompleted	Herhangi bir aksiyon adımı biter (tık, sürükle-bırak, ekipman giyme, quiz, kamera hareketi vb.)	SequenceManager.cs OnActionCompleted() (L570)	actionId, levelId, sequenceId, type**, objectId, startTime, endTime, duration, result
6	QuizAnswered	Oyuncu bir quiz seçeneği işaretler	UIQuizPanel.cs OnOptionClicked() (L171)	actionId, levelId, sequenceId, questionId***, selectedAnswer, correctAnswer, isCorrect, attempts, attemptIndex, timeSpent
7	QuizSummary	LogLevelCompleted sonunda, o seviyede en az bir quiz cevaplandıysa otomatik ateşlenir	PlayFabDataManager.cs LogQuizSummary() – private (L401)	levelId, totalQuestions, correctAnswers, wrongAnswers, accuracy
8	DragDropAttempt	Bir öge bir bırakma bölgesine başarıyla kabul edilir	UIDropZone.cs AcceptItem() (L201)	actionId, levelId, sequenceId, targetObject, attempts, placements[] {item, droppedOn, correct}
9	MistakeRecorded	(a) quiz yanlış cevap; (b) bırakma bölgesine yanlış öge bırakılması	SequenceManager.cs OnQuizFailed() (L686); UIDropZone.cs TryAcceptItem() (L179)	mistakeType, actionId, levelId, sequenceId, severity****, timestamp
10	SurveyCompleted	Saha anketi aksiyonu (dropdown Q&A + fotoğraf yakalama) tamamlanır	SurveyResultTracker.cs EndSession() (L187)	actionId, levelId, sequenceId, questionResults[], photoResults[], completionTime
11	SessionEnded	Giriş yapılmış durumda ve bir seviye devam ederken uygulama kapanır	PlayFabDataManager.cs OnApplicationQuit() (L594)	levelId

Kapsanmayan alanlar: ekonomi/para birimi, reklam, uygulama-içi satın alma, uygulama-açılış/kurulum olayları, PlayFab girişinin örtük olarak yakaladığının ötesinde cihaz meta verisi, veya bağımsız buton-tıklama telemetrisi (butonlar yalnızca bir eğitim aksiyonunu tamamlattıklarında `ActionCompleted{type:"click"}` üzerinden dolaylı olarak izlenir).

* `completionRate` gerçek bir tamamlanma yüzdesi değildir, bkz. Bölüm 7. ** 10 `ActionType` enum değeri 5 string'e indirgenir (`GetActionTypeString`, L1396). *** `questionId`, `actionId` ile aynı değeri taşır. **** `severity` her iki çağrı noktasında da sabit `1` literalidir.

6. Depolama / Kuyruklama / Yeniden-Deneme Mekanizmaları

- **Yalnızca bellek-içi giriş-öncesi kuyruk:** `PlayFabDataManager._pendingEvents` (`Queue<WriteClientPlayerEventRequest>`), `_isLoggedIn` false iken ateşlenen olayları biriktirir; başarılı girişte `FlushPendingEvents()` senkron olarak boşaltır. Bu kuyruk **kalıcı değildir** — giriş tamamlanmadan uygulama çökerse veya zorla kapatılırsa, kuyruktaki tüm olaylar kaybolur.
- **Çevrimdışı/disk ön belleği yok:** `Assets/_Project` içinde telemetri için hiçbir yerel veritabanı, dosya-tabanlı kuyruk veya `PlayerPrefs` kullanımı yoktur (proje genelinde grep ile doğrulandı — sıfır sonuç).
- **Gönderim hatasında yeniden deneme yok:** `SendEventRequest()` 'in hata callback'i yalnızca `Debug.LogError(...)` yapar; başarısız bir `WritePlayerEvent` çağrısı (ağ kopması, sunucu hatası) sessizce düşer — ne yeniden kuyruğa alma, ne üstel geri çekilme, ne de sonradan tekrar denemek için kalıcı kayıt vardır.
- **Batch (toplu gönderim) yok:** her `Log*` çağrısı tam olarak bir anlık `WritePlayerEvent` API çağrısına dönüşür (istemci-tarafli biriktirme penceresi yok) — bu nedenle yoğun etkileşim dizileri (ör. çok sayıda yanlış sürükle-bırak denemesi) çok sayıda küçük ayrı ağ çağrısı üretir.
- **Whitelist ön bellekleme:** `GetTitleData` ile çekilen çalışan whitelist'i (`PlayerEntry[]`) yalnızca bellekte tutulur (`UILoginPanel._entries`); her uygulama başlangıcında yeniden çekilir. PlayFab'a erişilemezse yerel ön bellek/fallback yoktur.
- **Paylaşılan okuyucu hesap hilesi:** PlayFab Development katmanının "cihaz başına 1000 hesap" kotasını tüketmemek için, gerçek çalışan girişinden önce Title Data okumak amacıyla sabit-kodlanmış paylaşılan bir `CustomId = "whitelist_reader"` hesabı kullanılır (`PlayFabDataManager.cs` L126-153, kod içi yorumlarla belgelenmiş).

7. Gizlilik / Onay (KVKK / GDPR) Durumu

Hiçbir gizlilik mekanizması bulunamadı. `Assets/_Project` içinde hiçbir yerde KVKK/GDPR bilgilendirme ekranı, onay (consent) anahtarı, opt-out bayrağı, veri-silme rutini veya gizlilikle ilgili herhangi bir kod yoktur (`GDPR|consent|Consent|opt-out|OptOut|privacy|Privacy` için grep — sıfır eşleşme). Oyuncu kimliği, şifresiz şekilde whitelist'e karşı eşleştirilen kendi-girdiği bir çalışan ID'sidir (`UILoginPanel.cs` kod içi yorumu açıkça "Şifre alanı yoktur" der) ve bu aynı ID hem PlayFab `CustomId` 'si hem de her olayın `employeeId` 'si olur — yani **tüm telemetri, anonimleştirme veya takma-adlandırma katmanı olmaksızın, doğrudan ve kalıcı olarak isimli bir çalışana atfedilebilir** (doğru/yanlış cevaplar, zamanlama, hatalar dahil).

8. Tespit Edilen Eksiklikler ve Tutarsızlıklar

Depoda bu tam telemetri hattının olağandışı derecede detaylı bir öz-denetimi mevcut: `Caner_UI/thundershock-kpi-wireframe/DATA_MAPPING.md` ve `PRODUCT_CONTEXT.md` (aşağı akış KPI-panosu prototipi için yazılmış). Bu denetimi mevcut koda karşı çapraz kontrol etmek çoğu bulgunun hâlâ geçerli olduğunu, birkaçının ise zaten düzeltildiğini gösteriyor:

- **severity her zaman sabit 1** : Hem `SequenceManager.OnQuizFailed` hem `UIDropZone.TryAcceptItem` çağrı noktasında — hiçbir yerde bir önem-derecesi ölçeği tanımlı değil, bu nedenle "kritik hata" raporlaması bugün yapısal olarak mümkün değil.
- **questionId == actionId**: (`UIQuizPanel.cs` L173, aynı değeri iki kez geçiyor) — bir aksiyonun birden fazla sorusu olsaydı ayırt edilemezlerdi.
- **ActionCompleted.type, 10 ActionType enum değerini 5 string'e sıkıştırıyor** (`SequenceManager.GetActionTypeString`, L1396-1409) — `CameraMove`, `ModalWindow` ve `Fade` ayırt edilemez şekilde hepsi `"interaction"` olarak raporlanıyor.
- **completionRate alanı yanıltıcı adlandırılmış** — aslında `Clamp01(1 - mistakes*0.05)`, yani bir hata-ceza katsayısı, gerçek bir tamamlanma yüzdesi değil; denetim dokümanına göre aşağı akışta hiçbir yerde tüketilmiyor.
- **actionId benzersizliği garanti değil** — denetim, üretim içeriğinde seviyeler arası en az bir yinelenen actionID bulmuş (`Anahtari_cevir_Copy`).
- **Seviyeler arası tutarsız levelId değerleri** — `level_1.asset` `"level 1"` kullanıyor; `level_2.asset` `"lv11"` kullanıyor; `level_3.asset` `"NewLevel"` kullanıyor (ayarlanmamış Unity varsayılanı) — seviyeye göre herhangi bir agregasyon için gerçek bir veri-kalitesi sorunu.
- **role artık her olay payload'ında mevcut** (`SendEvent()`, `documentPayload["role"]` ayarlıyor) — daha önce `PRODUCT_CONTEXT.md` 'de eksik olarak işaretlenmişti, mevcut koddaki düzeltilmiş; aynı şekilde `sessionId` (artık her `LogLevelStarted` 'da üretilip her payload'a ekleniyor) ve `SurveyCompleted` (denetim dokümanı başlangıçta anket verisinin PlayFab'a hiç ulaşmadığını işaretlemişti; artık `SurveyResultTracker.EndSession` üzerinden ulaşıyor) düzeltilmiş. Bu, `Caner_UI/thundershock-kpi-wireframe/` içindeki tasarım-denetimi markdown dokümanlarının mevcut `Assets/_Project` koduna göre **kısmen güncelliğini yitirmiş** olduğu ve yeniden doğrulanmadan tam yetkili kabul edilmemesi gerektiği anlamına geliyor.
- **Sunucu tarafı (CloudScript) doğrulama veya agregasyon yok** — tam PlayFab CloudScript SDK'sı projede mevcut olmasına rağmen, `Assets/_Project` içinde hiçbir şey onu çağırıyor; wireframe dokümanlarında tarif edilen tüm KPI hesaplaması, ham PlayFab olaylarına karşı tamamen istemci tarafında (bu depo dışındaki ayrı raporlama portalinde) yapılması bekleniyor.
- **Toplu gönderim, yeniden deneme veya kalıcı çevrimdışı kuyruk yok** — sahada (elektrik şalt sahaları, GDD'ye göre potansiyel olarak zayıf bağlantı) kullanılan bir eğitim uygulaması, yukarıda tarif edilen küçük bellek-ichi giriş-öncesi kuyruk dışında geçici ağ kaybına karşı hiçbir dayanıklılığa sahip değil; girişten sonra gönderilemeyen herhangi bir olay yalnızca bir konsol logu ile birlikte kayboluyor.
- **Gizlilik/onay arayüzü veya veri saklama/silme aracı yok** — buna rağmen şifresiz gerçek bir çalışan kimliğine bağlı, kişisel olarak tanımlanabilir performans verisi (doğru/yanlış cevaplar, zamanlama, hatalar) topluyor.
- **Unity Analytics paketi manifest.json'da tanımlı ama kodda hiç kullanılmıyor** — ölü bağımlılık, gerçek bir entegrasyon değil.
- **PlayFabDataManager, MENU.unity sahnesine bağlı** (tüm .unity dosyaları GUID araması ile doğrulandı) — herhangi bir seviye sahnesi önce `MENU.unity`'den geçmeden doğrudan yüklenirse (ör. test/QA sırasında), `PlayFabDataManager.Instance` null olur ve tüm `Log*` çağrıları her yerde kullanılan `?.` null-koşullu operatörü üzerinden sessizce hiçbir şey yapmaz — o oturum için kullanıcıya/test edene hiçbir uyarı gösterilmeden sessiz ve tam veri kaybına yol açar.